软 件 工 程

实验三：软件实现与构建

王睿阳 241880548

2026 年 11 月

项目：外卖点餐与配送系统（校园食光）

目录

一、实现概述 **1**

二、UML 到代码的转换 **2**

1 类图 → 领域模型 2

2 组件图 → 服务装配 2

3 时序图 → 提交与支付的实现 2

4 与设计的差异及回改 3

5 代码规模 3

三、大模型辅助开发过程 **6**

1 分工与工作流 6

2 主要使用记录 6

3 生成内容不符合预期时的处理 7

4 对辅助开发的思考 7

四、编译与运行结果 **7**

1 运行方式 7

2 主链路运行 8

3 自动化验证 8

五、Git 远程仓库管理 **8**

六、结束语 **9**

参考文献 **9**

一、实现概述

本项目延续实验一的需求分析与实验二的设计建模，选题不变：单校区外卖点餐与配送系统「校园食光」，覆盖顾客、商家、配送人员、平台管理员四类角色。实验三的任务是把实验二的 8 张 UML 图落地为可运行的软件。

开工前先定了三个技术决策：

一是**语言与界面**选 Python + PySide6 桌面应用。课程建议写桌面应用方便后续实验做分析测试；PySide6 的 QSS 样式可以直接复刻实验二 UI 稿的橙色视觉（主色 #FF6B2C、圆角 12px），界面观感与设计稿一致，省掉了重画原型的工作。

二是**存储**选 SQLite 单文件库。组件图基础设施层画的是 MySQL，但原型阶段引入数据库服务器得不偿失；做法是把存储全部收敛在仓储层（repositories）里，业务代码只面对实体对象，将来换数据库服务层不用动。测试则用内存库（`:memory:`），每个用例拿到互相隔离的全新状态。

三是**工作流**采用题目建议的"需求 → 存根 + 规约 → 代码"路径的变体，实际执行是"需求 → 领域模型 → 仓储 → 服务实现 + 端到端验收"。原因在 3.4 节说明。

整体架构按实验二组件图同进程落地，共四层：

层	代码位置	职责
客户端层	campus_eats/ui	四角色界面 + 登录，只经网关调服务
网关	services/gateway.py	会话管理、角色鉴权（REQ-001）
应用服务层	campus_eats/services	网关 + 八个领域服务 + 调度器 + 时钟
基础设施层	campus_eats/repositories	SQLite 仓储，对应组件图的数据库

四条贯穿实现的关键约定（与实验一需求编号对应）：金额一律以 int 存「分」，任何环节不出现浮点（REQ-022）；订单状态只允许 OrderService.transit_to 迁移，其他服务经窄接口回写（组件图注记）；库存按「菜品 + 日期」记总量/预占/已售三量（REQ-005）；提交订单携带一次性令牌，重复提交返回首次结果（REQ-024）；全系统时间经注入的 Clock 取得，禁止直接调 datetime.now()（REQ-027）。

最后一条在演示上有直接收益：Clock 有三个实现——真实时钟、60 倍加速的演示时钟、测试用手动拨针时钟。「30 分钟未支付取消」这类规则在 demo 模式下约 30 秒即可观察到，课堂上可以现场验收。

二、UML 到代码的转换

1 类图 → 领域模型

类图六个包逐一映射为 models/ 下的模块：用户与权限（account.py）、店铺与菜品（shop_dish.py）、购物车与订单（cart_order.py）、支付与退款（payment_refund.py）、配送（delivery.py）、审计与通知（audit.py），枚举集中在 enums.py。

转换中做了三类决定：

状态机显式化。 状态图的每条迁移登记为字典 LEGAL_TRANSITIONS，Order.transit_to 只查表放行，表外一律抛 IllegalTransitionError。部分退款按状态图设计做成"制作中"的自环——不改状态、只加退款记录，这条是画状态图时才想清楚的，实现时直接受益。

类图上画在实体上的方法，按实现职责搬家。 login/logout 需要访问仓储并做密码校验，从 Account 移到 UserService；骑手接单要联动配送任务与订单状态回写，从 Rider 移入 DispatchService。实体只保留纯状态判断，这符合数据类不碰基础设施的分层原则。

金额与库存的语义下沉到领域层。 yuan_to_fen 用字符串拆分 + 整数运算实现两位小数转换，不经过浮点；DailyStock.reserve 在可用量不足时返回 False 而不是部分预占，保证 REQ-005 的"并发不超卖"能在最小单元成立。

2 组件图 → 服务装配

组件图的 API 网关收口对应 gateway.py：登录建会话，require(*roles) 统一做角色鉴权，四个端的界面全部经它取当前账号后再调服务。八个领域服务与组件图一一对应；服务之间的箭头（ORD→PAY、DISP→ORD、AFTER→ORD/PAY）在 app.py 的装配段用注入衔接——服务之间不互相构造，避免环。出餐生成配送任务这条 ORD→DISP 的边，用的是回调装配：OrderService.on_order_ready 由 app 接到 DispatchService.create_task。

组件图右侧的定时任务调度对应 scheduler.py，驱动五条自动规则：30 分钟未支付取消并释放库存（REQ-010）、10 分钟未接单取消退款（REQ-015）、已送达 2 小时自动完成（REQ-013）、取消申请 24 小时未处理自动同意（REQ-026）、出餐 15 分钟无人接单转配送异常挂起（状态图「配送超时」），外加跨日库存重置（REQ-005，按「菜品+日期」惰性建行实现，新的一天首次访问即拿到重置后的份数）。调度器的 run_due() 与线程循环分离，测试里用 FakeClock 拨针后直接调用，超时规则因此可以被精确断言。

3 时序图 → 提交与支付的实现

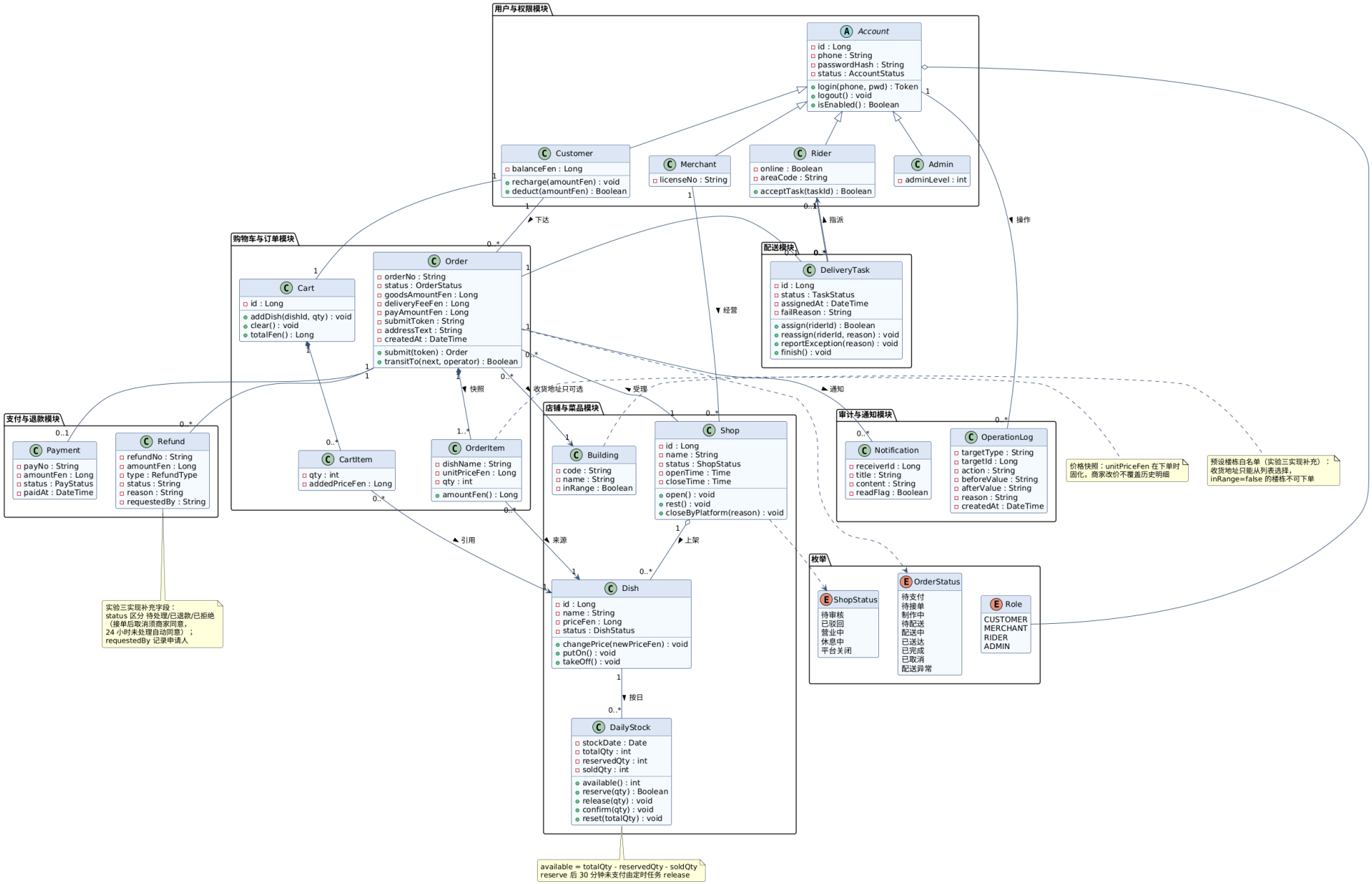
时序图「下单与支付」的三个 alt 分支逐条落地在 OrderService.submit_order 与 PaymentService.pay：令牌已使用返回首次订单；库存不足整单失败并回滚已预占的份数（多行购物车逐行预占、失败即逆序释放）；余额不足抛 InsufficientBalanceError 且订单保持待支付。重复支付命中已有流水时直接返回首次结果，不产生第二条扣款。

4 与设计的差异及回改

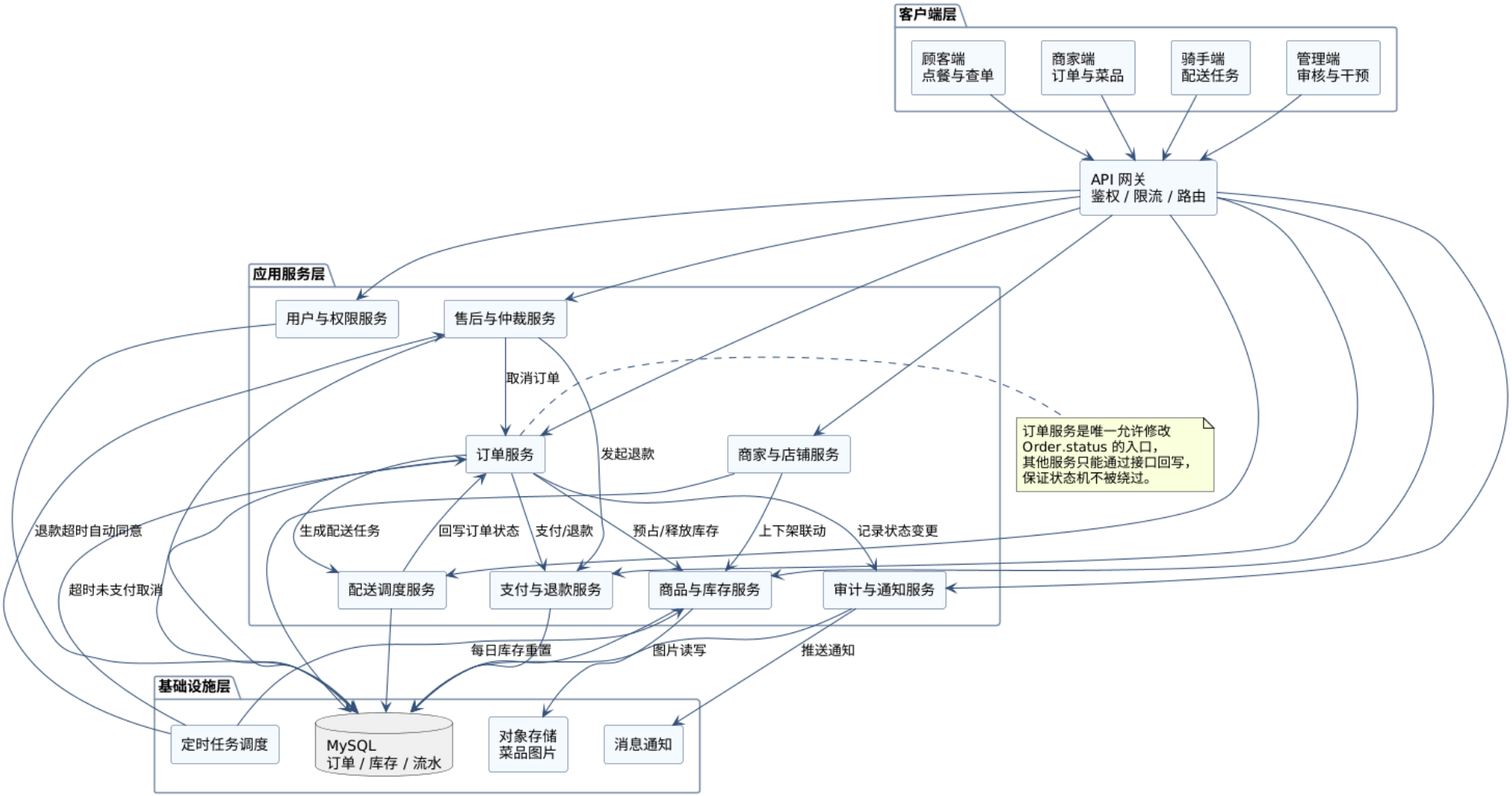
实现与实验二设计有五处出入，按实验二报告第 6 章的约定"以实现为准回改设计"，类图已更新并重新渲染（提交 7949c53）：

1. OrderStatus 补「配送异常」挂起态——状态图里有、类图枚举漏了
2. ShopStatus 补「待审核 / 已驳回」前置态——REQ-002 的入驻审核流程需要
3. 新增 Building 预设楼栋类——REQ-008 要求地址只能从白名单选择
4. Refund 补 status / requestedBy 字段——"24 小时未处理自动同意"需要退款状态机
5. login / acceptTask 两个方法从实体移入服务（见 2.1 节）

5 代码规模


用户与权限模块
Account
id : Long
phone : String
passwordHash : String
status : AccountStatus
login(phone, pwd) : Token
logout() : void
isEnabled() : Boolean
Customer
balanceFen : Long
recharge(amountFen) : void
deduct(amountFen) : Boolean
Merchant
licenseNo : String
Rider
online : Boolean
areaCode : String
acceptTask(taskId) : Boolean
Admin
adminLevel : int
下达
经营
指派
操作
购物车与订单模块
Cart
id : Long
addDish(dishId, qty) : void
clear() : void
totalFen() : Long
Order
orderNo : String
status : OrderStatus
goodsAmountFen : Long
deliveryFeeFen : Long
payAmountFen : Long
submitToken : String
addressText : String
createdAt : DateTime
submit(token) : Order
transitTo(next, operator) : Boolean
配送模块
DeliveryTask
id : Long
status : TaskStatus
assignedAt : DateTime
failReason : String
assign(riderId) : Boolean
reassign(riderId, reason) : void
reportException(reason) : void
finish() : void
快照
收货地址只可选
受理
通知
支付与退款模块
Payment
payNo : String
amountFen : Long
status : PayStatus
paidAt : DateTime
Refund
refundNo : String
amountFen : Long
type : RefundType
status : String
reason : String
requestedBy : String
CartItem
qty : int
addedPriceFen : Long
OrderItem
dishName : String
unitPriceFen : Long
qty : int
amountFen() : Long
店铺与菜品模块
Building
code : String
name : String
inRange : Boolean
Shop
id : Long
name : String
status : ShopStatus
openTime : Time
closeTime : Time
open() : void
rest() : void
closeByPlatform(reason) : void
审计与通知模块
Notification
receiverId : Long
title : String
content : String
readFlag : Boolean
OperationLog
targetType : String
targetId : Long
action : String
beforeValue : String
afterValue : String
reason : String
createdAt : DateTime
价格快照：unitPriceFen 在下单时固化，商家改价不覆盖历史明细
预设楼栋白名单（实验三实现补充）：收货地址只能从列表选择，inRange=false 的楼栋不可下单
引用
来源
上架
枚举
Dish
id : Long
name : String
priceFen : Long
status : DishStatus
changePrice(newPriceFen) : void
putOn() : void
takeOff() : void
ShopStatus
待审核
已驳回
营业中
休息中
平台关闭
OrderStatus
待支付
待接单
制作中
待配送
配送中
已送达
已完成
已取消
配送异常
Role
CUSTOMER
MERCHANT
RIDER
ADMIN
按日
DailyStock
stockDate : Date
totalQty : int
reservedQty : int
soldQty : int
available() : int
reserve(qty) : Boolean
release(qty) : void
confirm(qty) : void
reset(totalQty) : void
实验三实现补充字段：
status 区分 待处理/已退款/已拒绝（接单后取消须商家同意，24 小时未处理自动同意）；
requestedBy 记录申请人
available = totalQty - reservedQty - soldQty
reserve 后 30 分钟未支付由定时任务 release

UML类图


客户端层
顾客端
点餐与查单
商家端
订单与菜品
骑手端
配送任务
管理端
审核与干预
API 网关
鉴权 / 限流 / 路由
应用服务层
用户与权限服务
售后与仲裁服务
取消订单
订单服务
发起退款
商家与店铺服务
订单服务是唯一允许修改
Order.status 的入口，
其他服务只能通过接口回写，
保证状态机不被绕过。
退款超时自动同意
生成配送任务
回写订单状态
支付/退款
预占/释放库存
上下架联动
记录状态变更
超时未支付取消
配送调度服务
支付与退款服务
商品与库存服务
审计与通知服务
每日库存重置
图片读写
推送通知
基础设施层
定时任务调度
MySQL
订单 / 库存 / 流水
对象存储
菜品图片
消息通知

UML组件图

规模统计（build/count_lines.py，口径：代码行 = 含非注释 token 的物理行去重）：

目录	文件数	总行数	代码行
campus_eats	44	5789	4417
tests	7	868	717
scripts	1	282	231
合计	52	6939	5365

满足"大于 1500 行"的要求，其中不含任何 GUI 设计工具生成物——界面全部为手写 QSS 与布局代码。

三、大模型辅助开发过程

1 分工与工作流

本次实验使用 ZCode（CLI Agent，模型 GLM）。分工与实验二一脉相承：**我负责拍板与验收节奏**——三个技术决策、每层完成后的推进判断、验收标准的取舍；**模型承担实现主体**——从领域模型、仓储、八个服务到 PySide6 界面与测试的代码产出；**自动检查链兜底**——pylint 静态检查、离屏界面冒烟、32 项端到端验收、63 项 pytest，每层产物先过检查再进下一层。

全程的真实过程记录在仓库 lab3/ai_log.md，按"交给模型的任务 / 模型产出 / 发现的问题 / 处理"四要素写了六条，下文引用的事件均可在其中查到出处。

2 主要使用记录

记录一：实现计划与选型。 把题目、27 条需求、8 张 UML 源码交给模型要求给出计划并列出需要人工拍板的决策点。模型给出了 14 步计划并收敛出三个决策问题（界面框架、存储、工作流），与我的判断一致。其中它最初把"每步暂停等人工审查"写进计划——14 步暂停 14 次不现实，我改成只在关键节点推进。这条也定了整个实验的节奏：决策在人，推进看验收结果。

记录二：pylint 配置的环境核对。 模型直接手写 .pylintrc，但本机装的是 pylint 4.1，配置节名与旧版不同（如 [MASTER] 已改名），手写配置可能静默失效。处理办法是先跑 pylint --generate-rcfile 生成参考配置、核对节名与选项名之后再写自定义项。这件事确认了一个工作习惯：模型给的配置文件和代码一样，必须验证过再采用。

记录三：陈旧实体覆盖状态。 服务层变异方法最初接收调用方传来的 Order 实体。端到端验收跑到"部分退款"时失败——退款服务改的是数据库里的副本，测试手里的本地引用还是旧状态。根因是"状态存在数据库、内存对象会过期"这类跨模块一致性，模型写单点代码时不会主动考虑。重构方案是变异方法一律按 order_id 现取最新实体再迁移，波及四个服务的全部调用点，逐个核对后通过。

记录四：测试套件反向暴露服务缺陷。 把手工验收脚本收编为 pytest 时，两个用例跑红暴露了真实缺陷：骑手完成单统计走了"未完任务"查询导致永远为 0；营业额把未支付订单也计了进去。这轮让我确认测试的价值

在"跑红 → 修服务 → 跑绿"的循环里，而不在"全绿"本身。

3 生成内容不符合预期时的处理

十次值得记录的返工里，问题被三种手段分别抓住，边界很清晰：

静态检查抓得住的：money.py 漏 import 自己引用的异常类（pylint E0602）、字典与括号风格告警。这类问题在"写完一层立即 lint"的流程下当场修复，成本最低。

跑起来才暴露的：配送任务的状态迁移方法 _transit 只做合法性校验、从不赋值 status——类型正确、逻辑缺失，静态检查完全无感，是端到端验收第 15 步"骑手拒单回待分配"失败才暴露的；同类还有四类账号子类没有重设 role 默认值（构造 Rider 得到的 role 是 CUSTOMER），靠四角色全链路冒烟抓出。

只有读语义才发现的：令牌幂等在首单提交后购物车已清空，重试时内容指纹必然失配——修法是明确语义"空购物车视为对上一单的重试，内容变化才算重放"；更典型的是一个假通过的测试用例：想测令牌重放，却生成了新令牌，等于没测。它跑绿了，是我读断言语义才发现它什么都没断言到。

处理方式上有一条贯穿的规律：模型的问题从来不是修不动，而是它不知道自己错——所以每次返工都同时把"这类错怎么被自动发现"补进检查链，同类问题不出现第二次。

4 对辅助开发的思考

与实验二"模型是打字员"的结论相比，这次分工往前走了一步：在状态机、金额口径、幂等语义这些需求已经写死的区域，模型可以直接翻译且一次通过率高；但在"部分退款后整单取消应只退剩余额度"这种需求没写明、需要口径决策的地方，它不会停下来问，需要一个明确的外部判断。所以实际工作流不是题目提示的"需求→存根+规约→代码"，而是"需求→实现→验收规约后置到端到端断言"——存根阶段被跳过的原因是存根的类型标注依赖仓储类型，而仓储必须先有；规约没有消失，变成了 32 项验收断言。这个调整是否值得，报告的结论是：对单人课程项目，能跑的验收比接口清单更能兜住质量，但若团队并行开发，接口先行仍是必要的。

四、编译与运行结果

1 运行方式

环境：Python 3.10.11，依赖 PySide6 / pytest / pylint（见 requirements.txt）。

```powershell
cd lab3
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\python -m campus_eats --demo
```

--demo 为演示模式，时间 60 倍加速。首次运行自动建库并写入种子数据：2 家店铺、6 个菜品、3 名骑手、7 个预设楼栋（含 1 个配送范围外楼栋用于 REQ-008 验收）。演示账号：顾客 13800000001、商家 13900000001、骑手 13700000001、管理员 13600000001，密码均为 123456。

2 主链路运行

【截图占位 1：登录页——登录后按角色进入对应端】

【截图占位 2：顾客端店铺详情——菜品行显示剩余份数与售罄标记】

【截图占位 3：确认订单——楼栋下拉限选、价格锁定说明】

【截图占位 4：商家端订单工作台——待接单/制作中/履约中三分区】

【截图占位 5：管理端派单与干预——操作记录表】

主链路在 demo 模式下可完整走通：浏览加购 → 提交订单（预占库存并锁价）→ 余额支付 → 商家接单出餐 → 管理员按片区派单 → 骑手接受并送达 → 顾客确认收货。超时规则可现场观察：待支付订单放置约 30 秒被自动取消并回补库存；支付后商家不接单，约 10 秒自动取消并全额退款。

3 自动化验证

验证项	命令	结果
单元与集成测试	python -m pytest	63 passed
代码风格	python -m pylint campus_eats	10.00/10
端到端验收	python scripts/e2e_check.py	32 项全过
并发不超卖（REQ-023）	200 线程抢库存 10	恰好 10 单，库存归零不为负

并发用例用 200 个线程同时提交，断言成功订单恰好等于库存份数——这是实验一 REQ-023 的验收标准在原型上的直接落地。

五、Git 远程仓库管理

仓库延续实验一、二：github.com/marcus-37/lab1-takeout，main 分支，本次推送区间 ba7e9b2 → 1029c55。实验三放在仓库 lab3/ 子目录，实验一要求"每人建立自己的公开仓库"，二三未另行要求，因此不另建仓库。

提交按层划分，每个提交对应一次可验证的完成：

提交	内容
d8bd71f	脚手架：依赖清单、包骨架、pylint 配置、AI 记录

提交	内容
4906424	领域模型层：类图六包映射、状态机迁移表
c8fba63	SQLite 仓储层（含子类 role 缺陷修复）
a7a35c4	应用服务层：网关、八服务、调度器，e2e 32 项通过
b2f5278	PySide6 界面：登录 + 四角色端
fe3ae0d	pytest 套件 63 项 + 两个统计缺陷修复
7949c53	按实现回改实验二类图并重渲染
1029c55	报告管线、统计、打包脚本

两点说明：一是实验一固定的 lab1-ai-draft / lab1-final 标签全程未动；二是类图回改单独提交且落在 lab2 目录——实验二报告约定"以实现为准回改设计"，这个提交就是那次回改的记录，与实现提交分开，历史可追溯。

六、结束语

【请务必用自己的话改写本节：两三句即可——这轮实验里你对"人 + 模型 + 检查链"这套分工最真实的体会，以及一处你认为草稿没写到的东西。】

参考文献

[1] 软件工程课程. 实验三：软件实现与构建（实验指导书）. 2026.

[2] 校园食光项目. requirements.md：实验一需求规格（v1.0）. 2026. 仓库 lab1-takeout 根目录.

[3] 王睿阳. 实验二报告：软件设计与建模. 2026. 仓库 lab1-takeout lab2 目录.

[4] 校园食光项目. ai_log.md：实验三大模型辅助开发记录. 2026. 仓库 lab1-takeout lab3 目录.